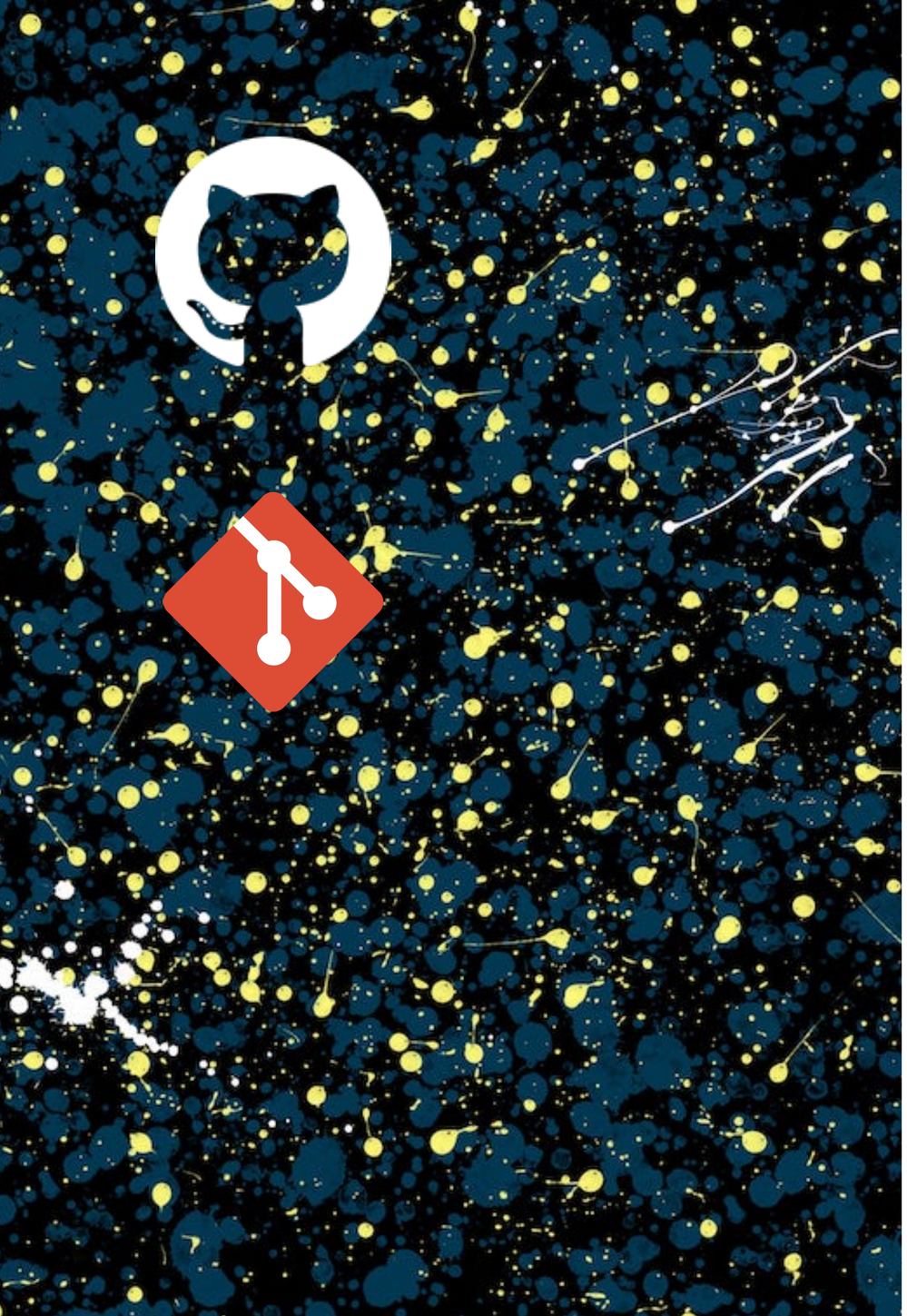
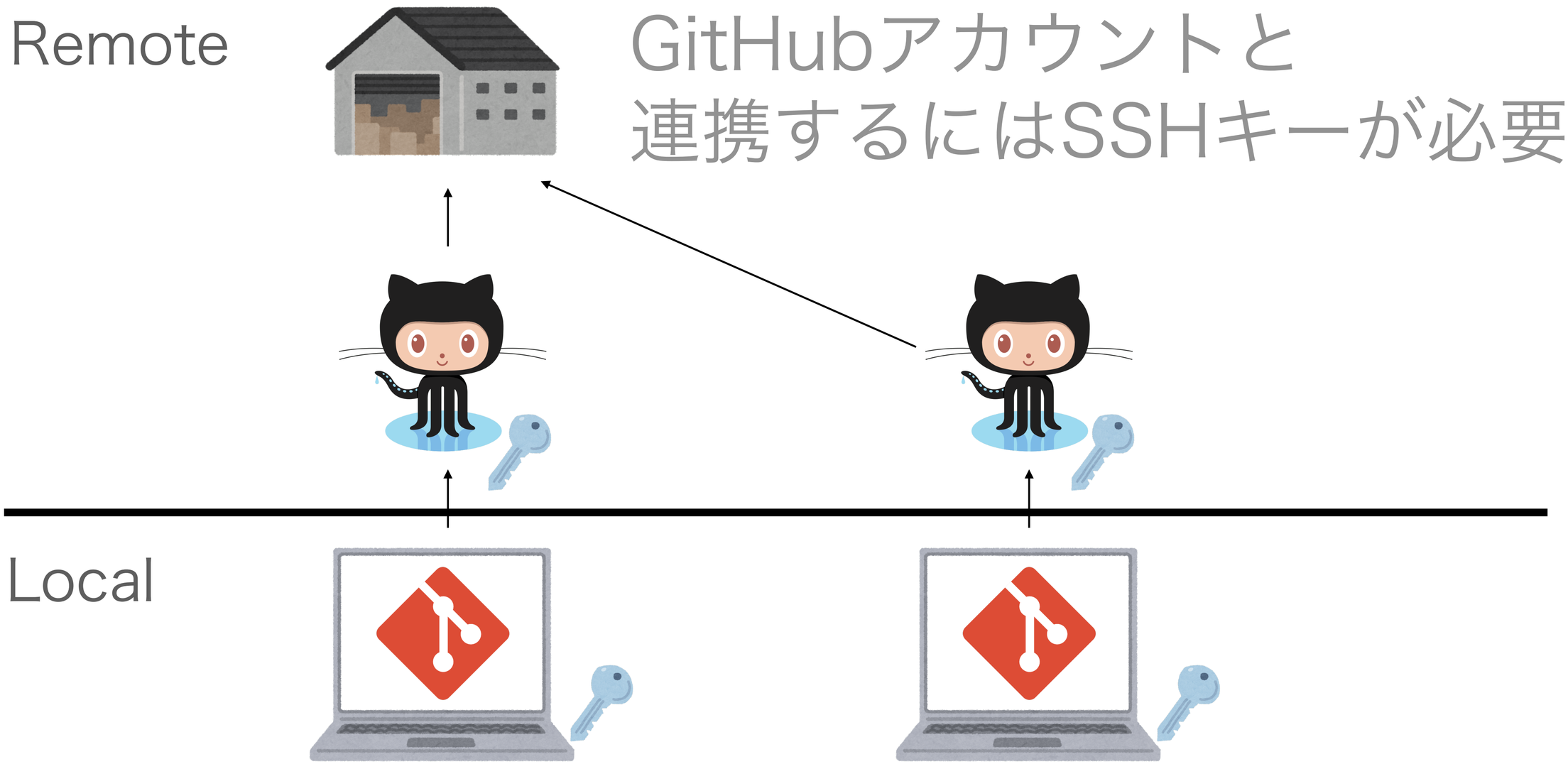
チーム開発
vol. 1

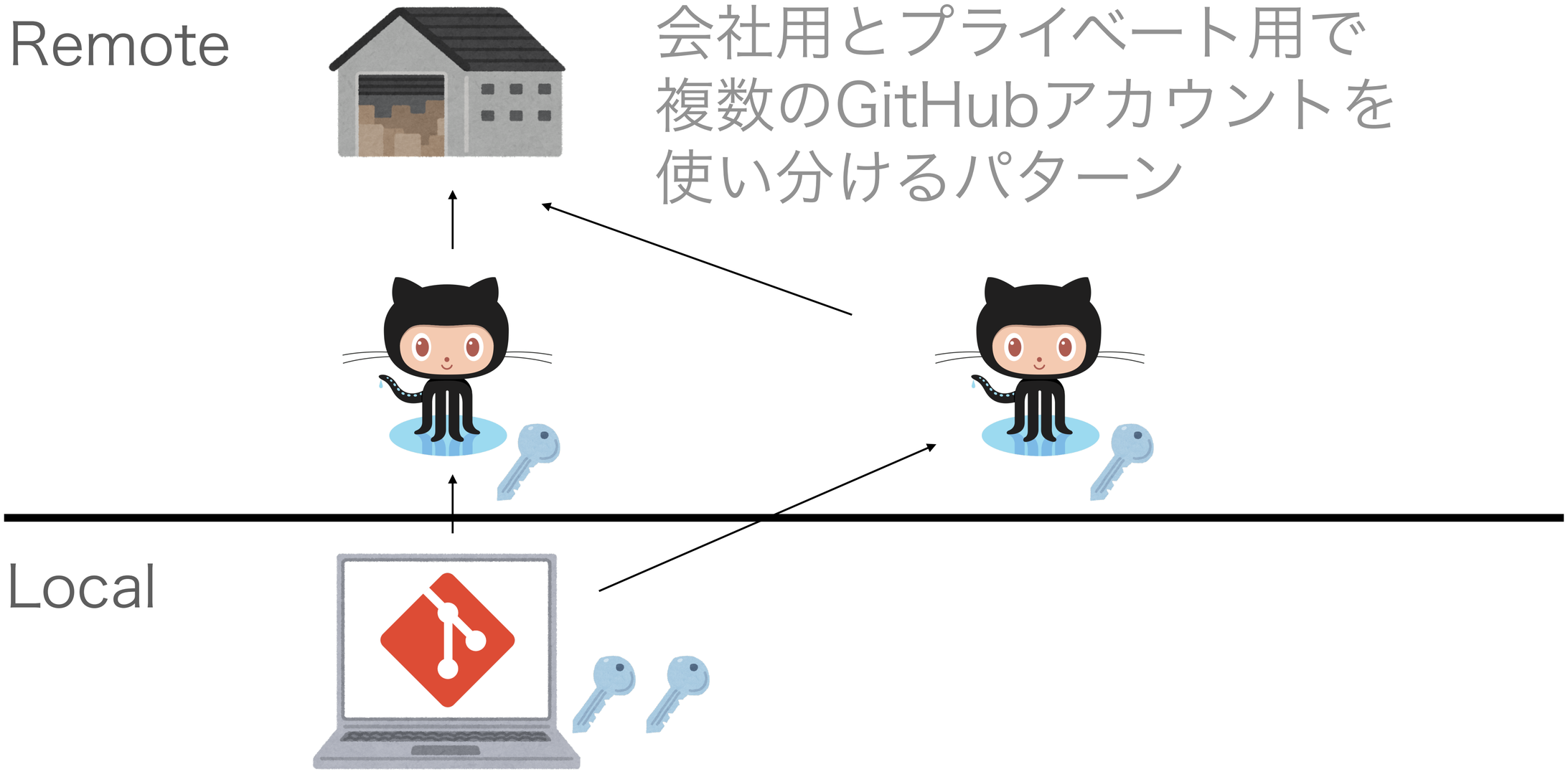


GitHub
複数アカウント設定

複数GitHubアカウント



非推奨:1つのPCに2つのSSHキー



GitHubとしては非推奨

GitHub利用規約
B. アカウント利用規約

1人の個人または1つの法人が複数の無料「アカウント」を保持することはできません

https://docs.github.com/ja/site-policy/github-terms/github-terms-of-service#b-account-terms

マルチアカウントは自己責任で

ZOZO
全社的に会社用GitHubアカウントを廃止した件
https://techblog.zozo.com/entry/github_sso

当講座としては
設定後トラブルがあった場合、
責任は持ちかねますので、
自己責任という事でお願いいたします

複数アカウントの流れ

1.サブ用のメールアドレスで
　GitHubにアカウントを作成する

2. パソコンでサブ用のsshキー作成

3. ~/.ssh/config ファイル作成し追記

4. GitHub(サブアカウント)に公開鍵を登録する

5. 疎通確認

6. 運用時の設定 (gitconfig, clone など)

2. サブ用のsshキーの作成

サブ用のSSHキーを作成
ssh-keygen -t ed25519 -C "メールアドレス" -f "id_ed25519_sub"

-fでSSHキーの名前指定

3. ssh設定ファイルに追記

```
~/.ssh/config (なければ作成)
Host github.com # メインアカウント
  HostName github.com
  User git
  Port 22
  IdentityFile ~/.ssh/id_ed25519  # メインアカウント用の鍵
  TCPKeepAlive yes
  IdentitiesOnly yes

Host github.com.sub # サブアカウント
  HostName github.com
  User git
  Port 22
  IdentityFile ~/.ssh/id_ed25519_sub    # サブアカウント用の鍵
  TCPKeepAlive yes
  IdentitiesOnly yes
```

4.5. 公開鍵登録 & 疎通確認

4. GitHubに公開鍵を登録する

5. 疎通確認 (configのHostに設定した名前)
ssh -T git@github.com
ssh -T git@github.com.sub

6. 運用 gitconfigの設定

普段使う方は global
リポジトリ毎に切り替える場合は local

cd リポジトリ名
git config --local user.name ユーザー名
 git config --local user.name メールアドレス

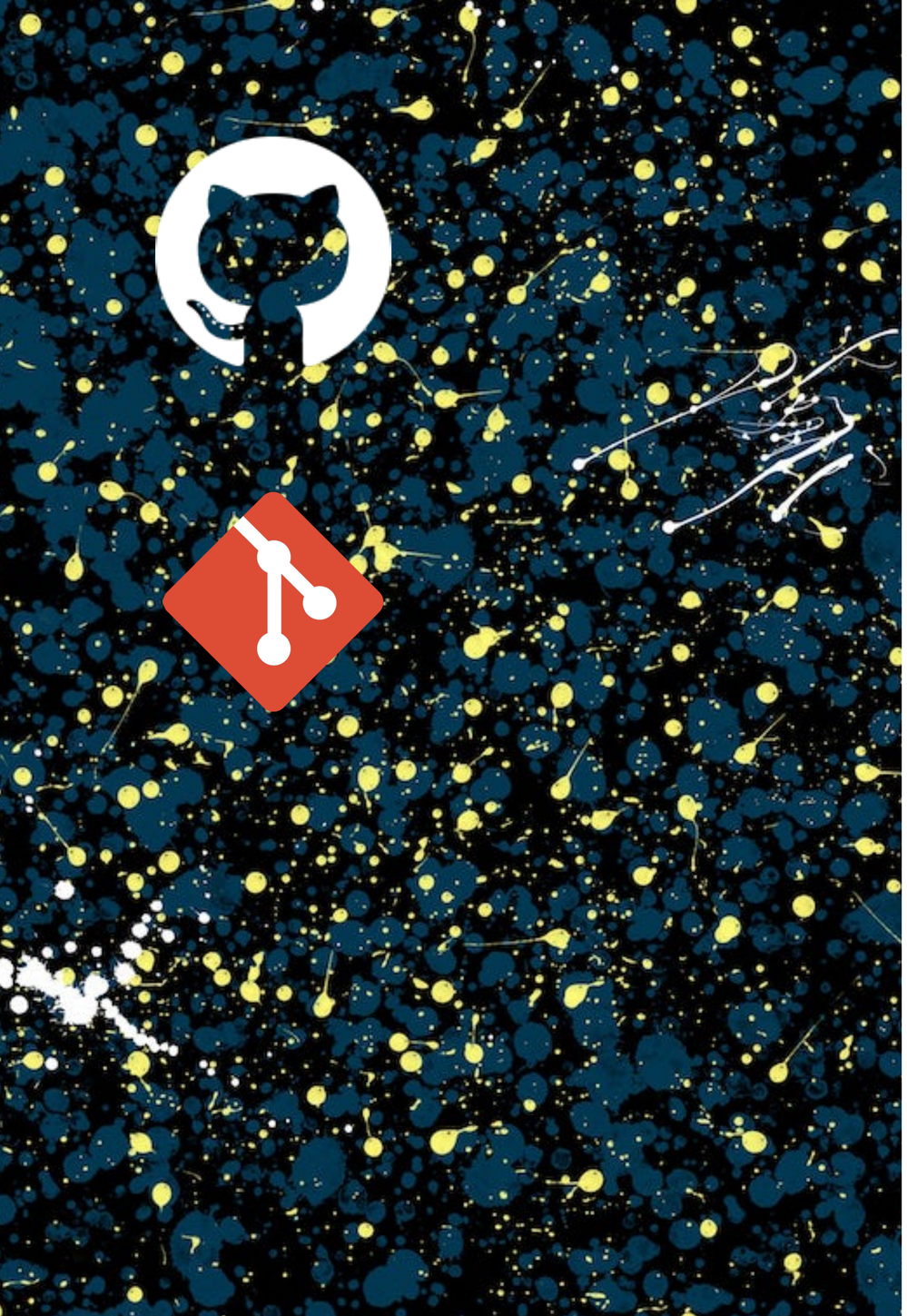
6. 運用 cloneする場合

// configに設定したHostを指定する

git clone git@github.com.sub:username/repository-name.git

git remote add origin git@github.com.sub:username/repository-name.git

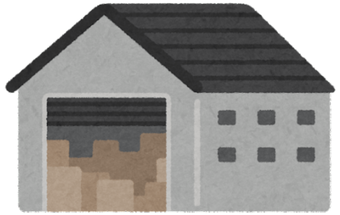
// git のurl確認
git config remote.origin.url

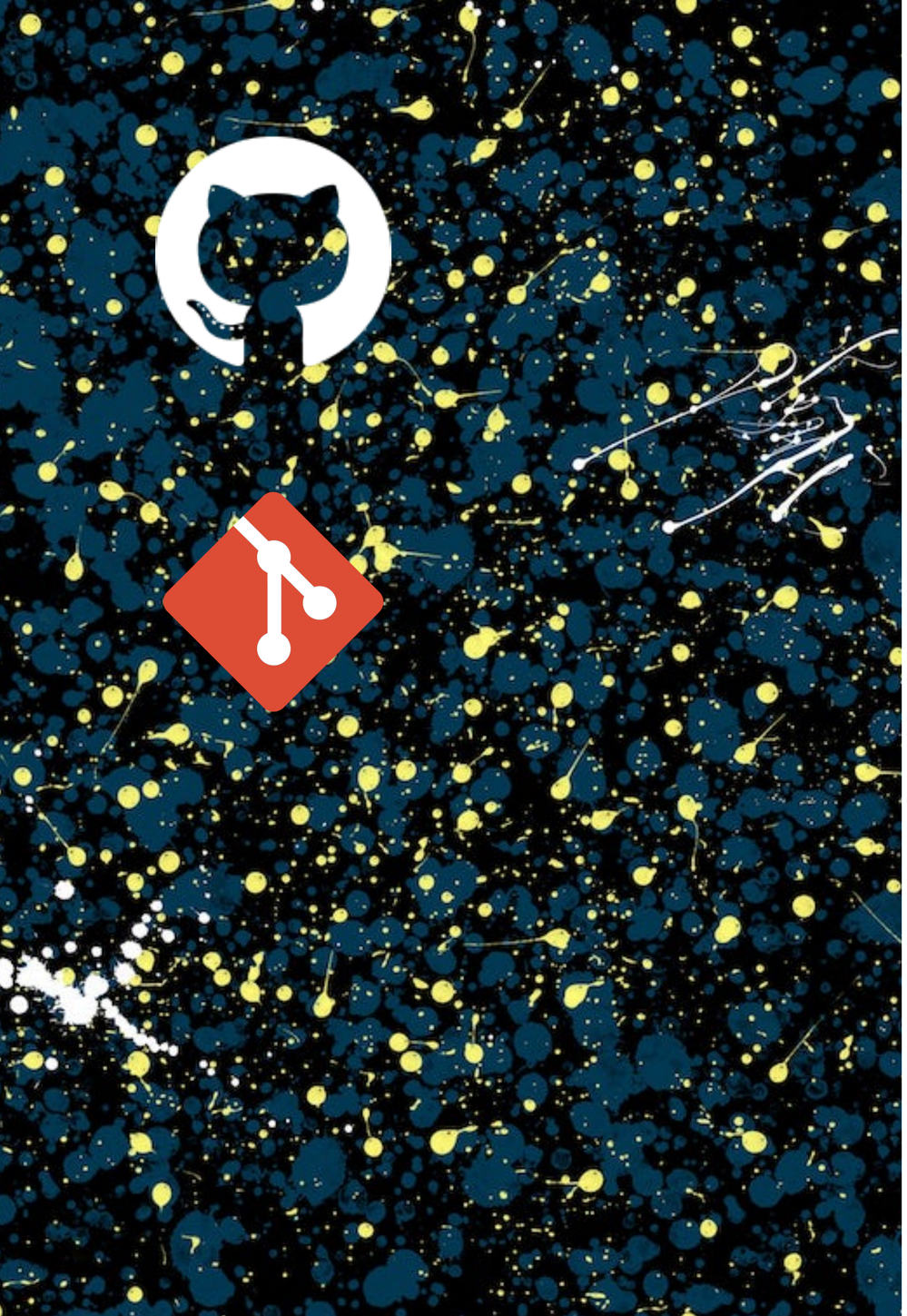


チーム開発用
リポジトリ作成

チーム開発用リポジトリ作成

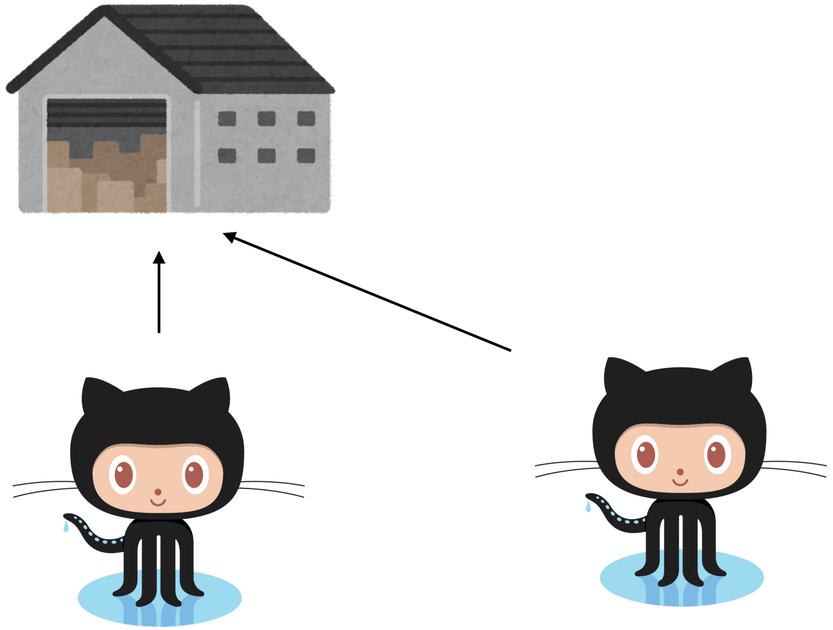
サブアカウントは削除する可能性があるため
メインアカウントの方に
チーム用のリポジトリをつくります。
(まずはprivateで作成します。)



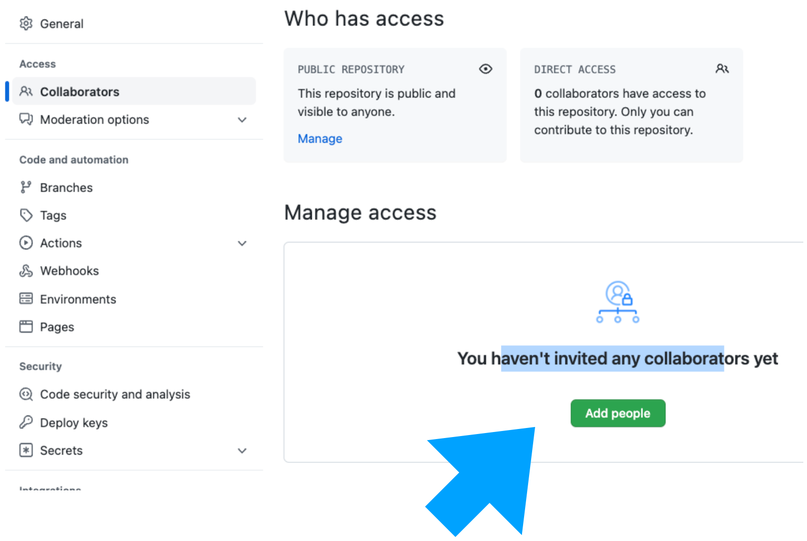


ユーザーの招待

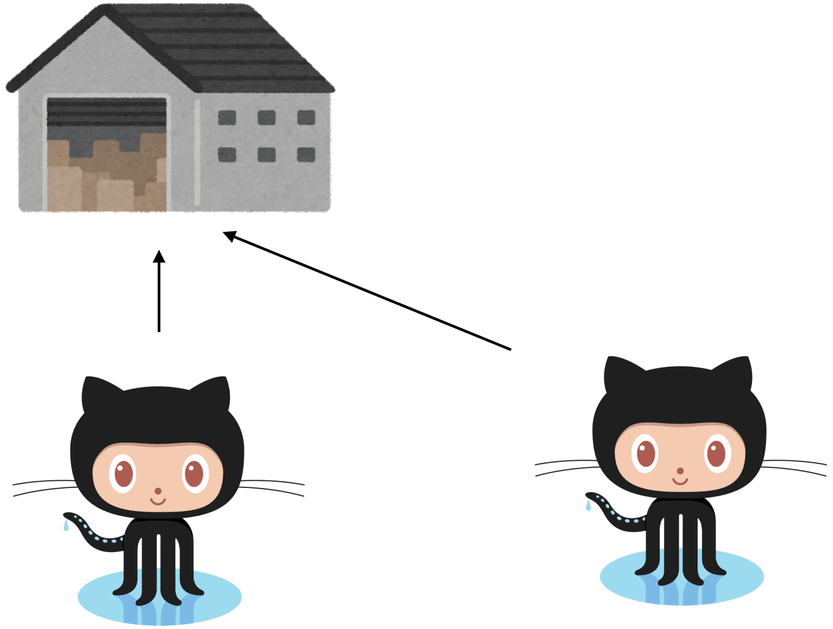
ユーザーの招待



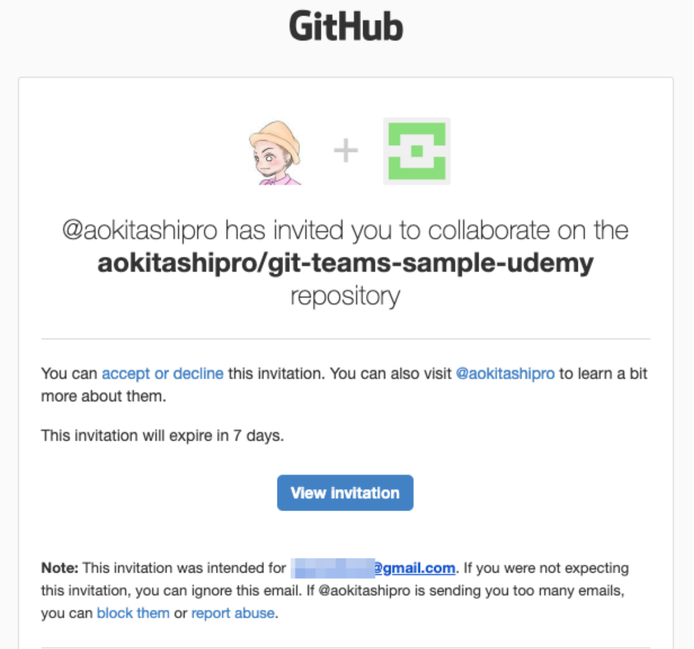
Settings-> Collaboratorsで
Add people で
招待したい人の
アカウント名などを指定すると
招待メールが飛びます。

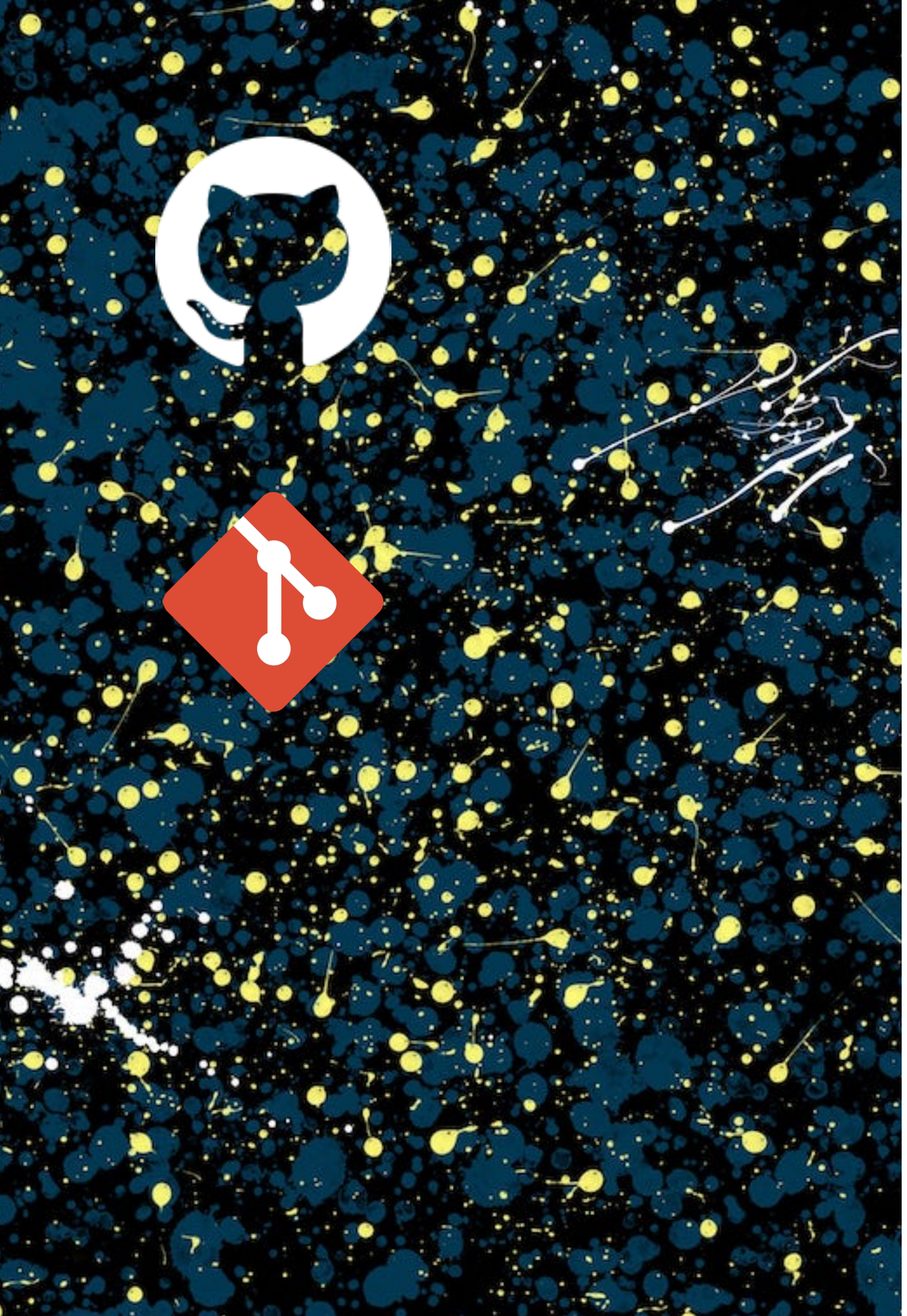


ユーザーの招待



招待された人が
受け入れると
privateリポジトリでも
表示されるようになります。

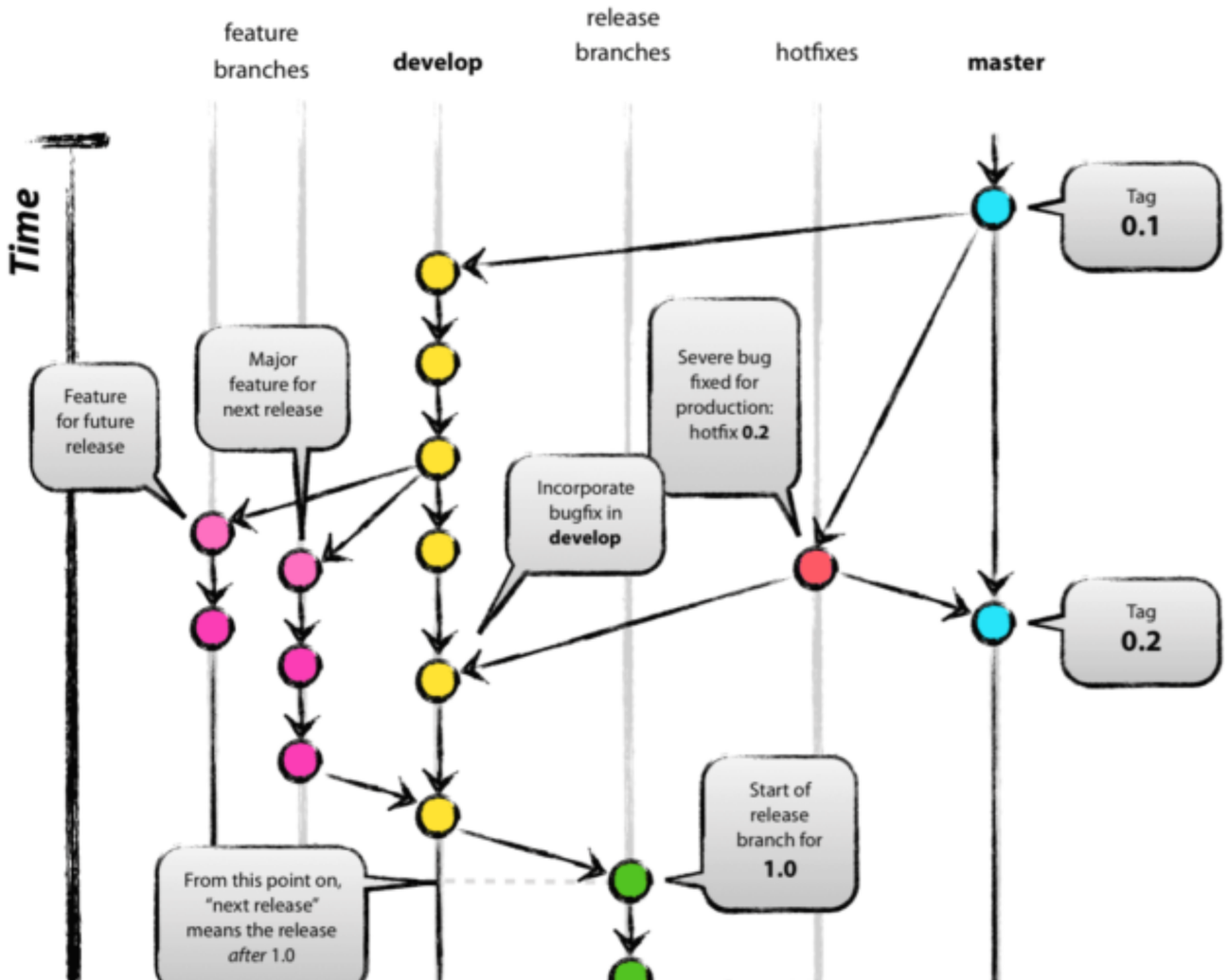




Git Flow

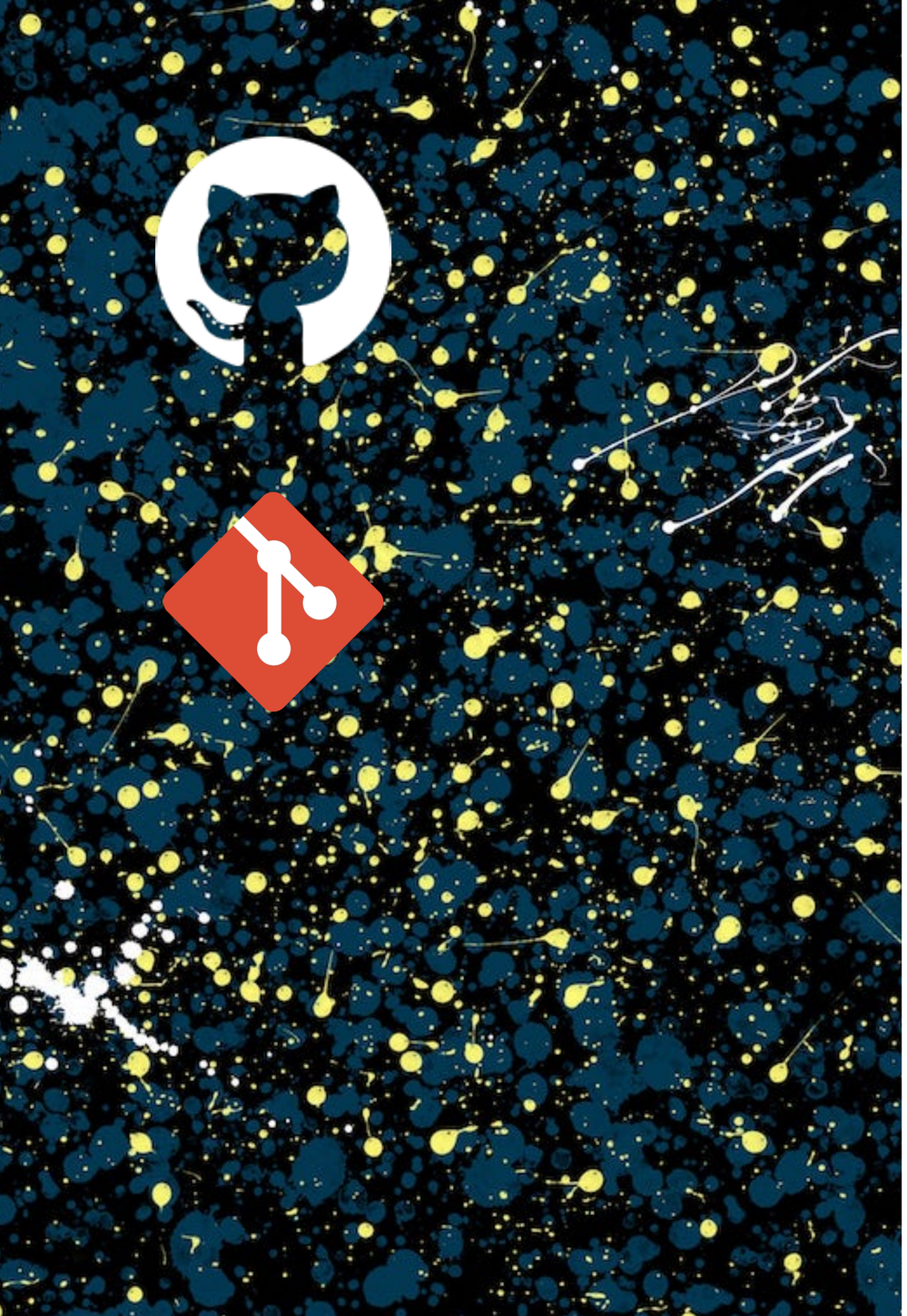
Git Flow

チーム開発におすすめのブランチモデル



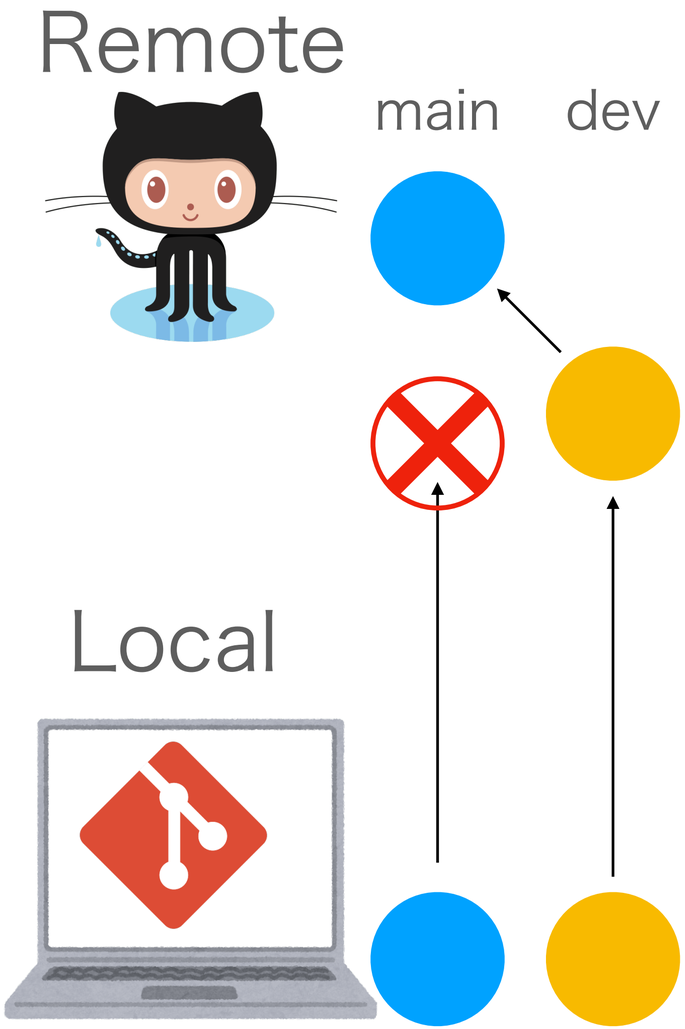
main
　リリース用
develop
　開発用

hotfixes
　バグ修正
release
　リリース手前のテスト用
feature
　新機能



直接pushを禁止
(privateの場合は
有料)

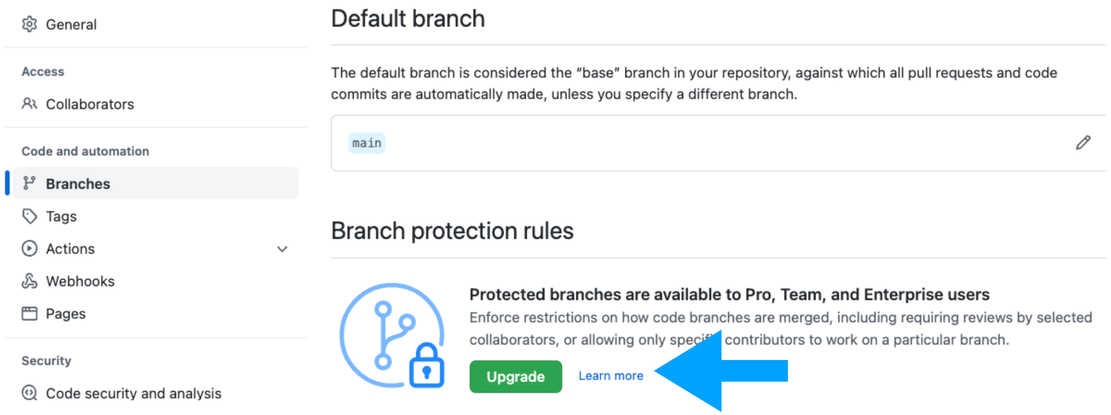
直接pushの禁止

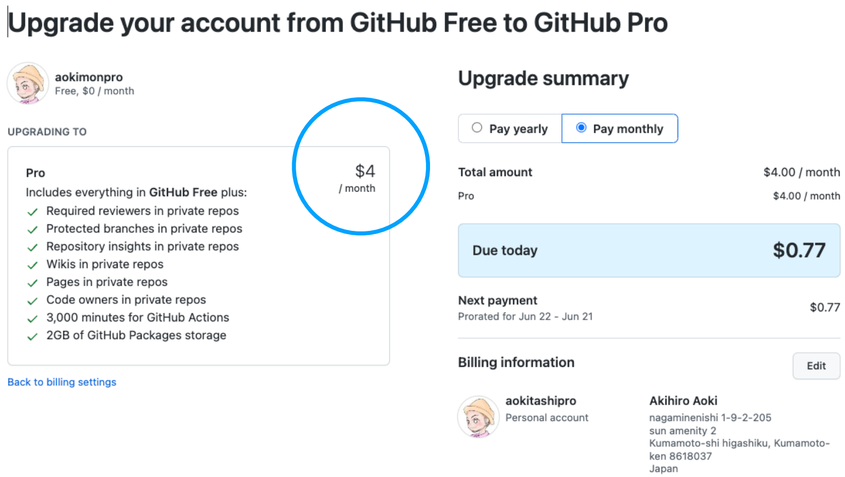


・ブランチの保護

Settings -> Branches->protection rules

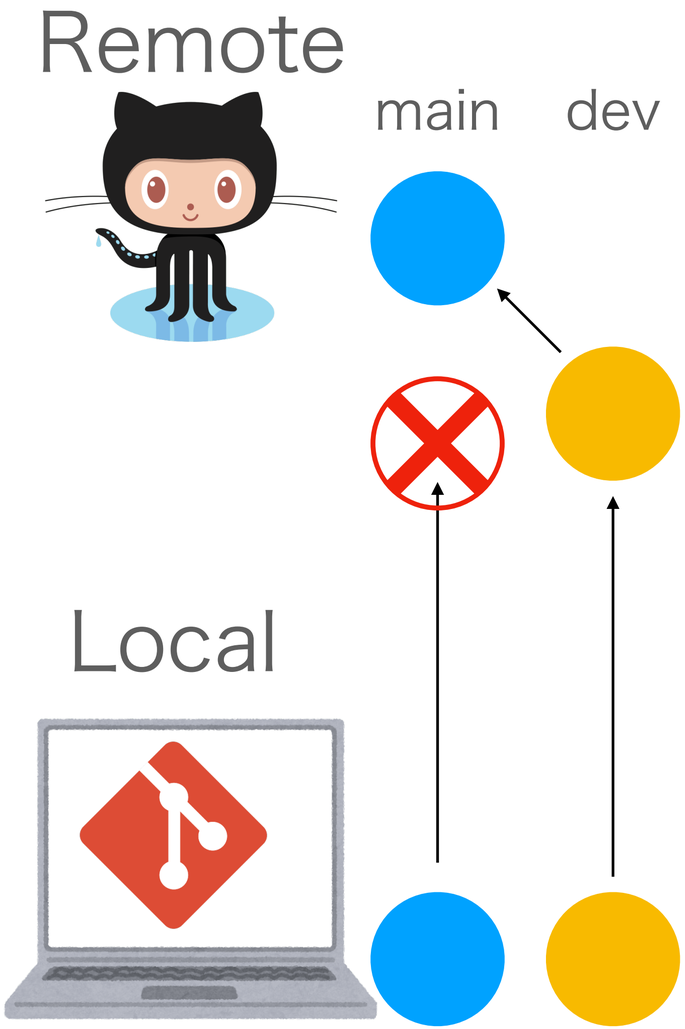
・レビューまたはステータスチェックを必須にする

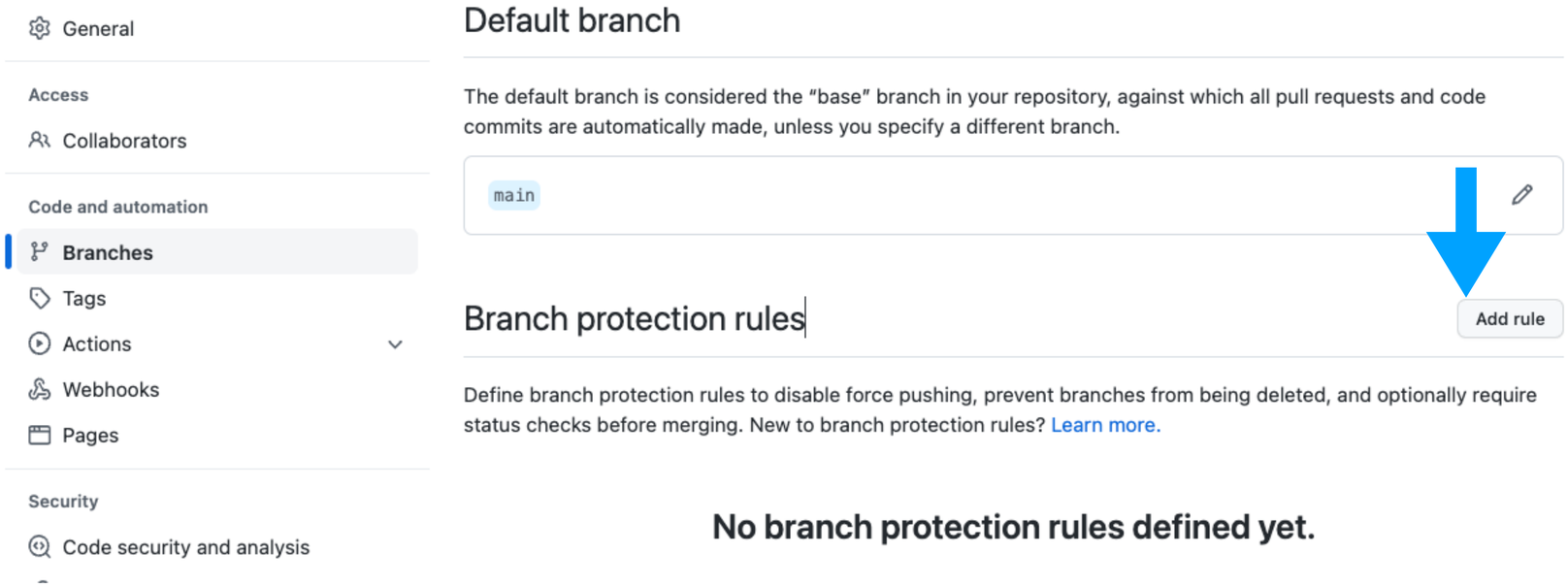




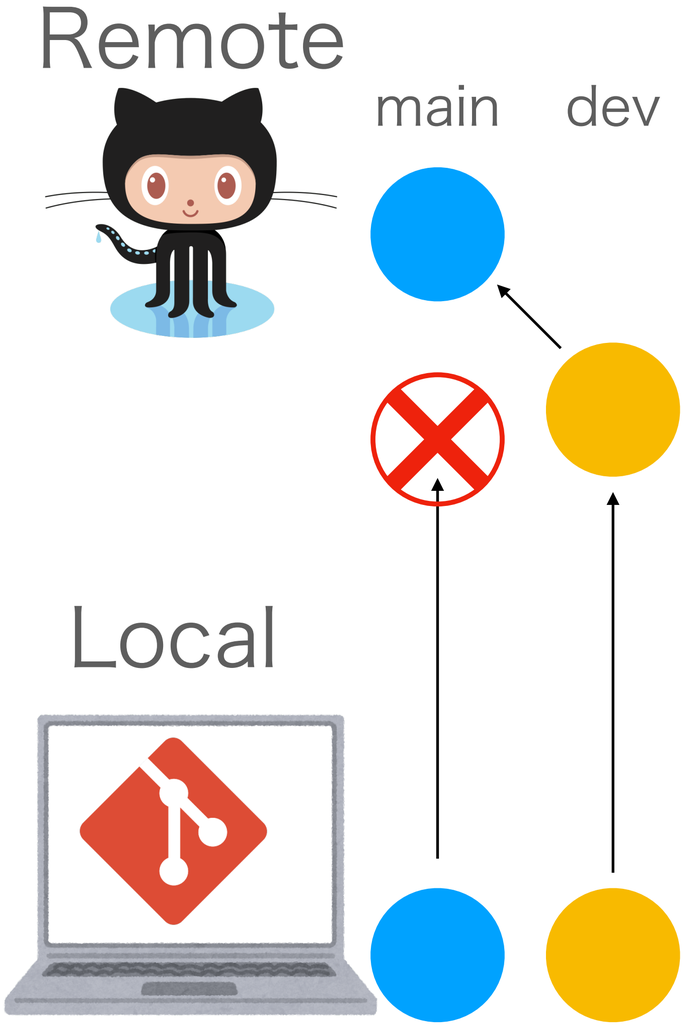
直接pushの禁止

・ブランチの保護
Settings -> Branches->protection rules
・レビューまたはステータスチェックを必須にする





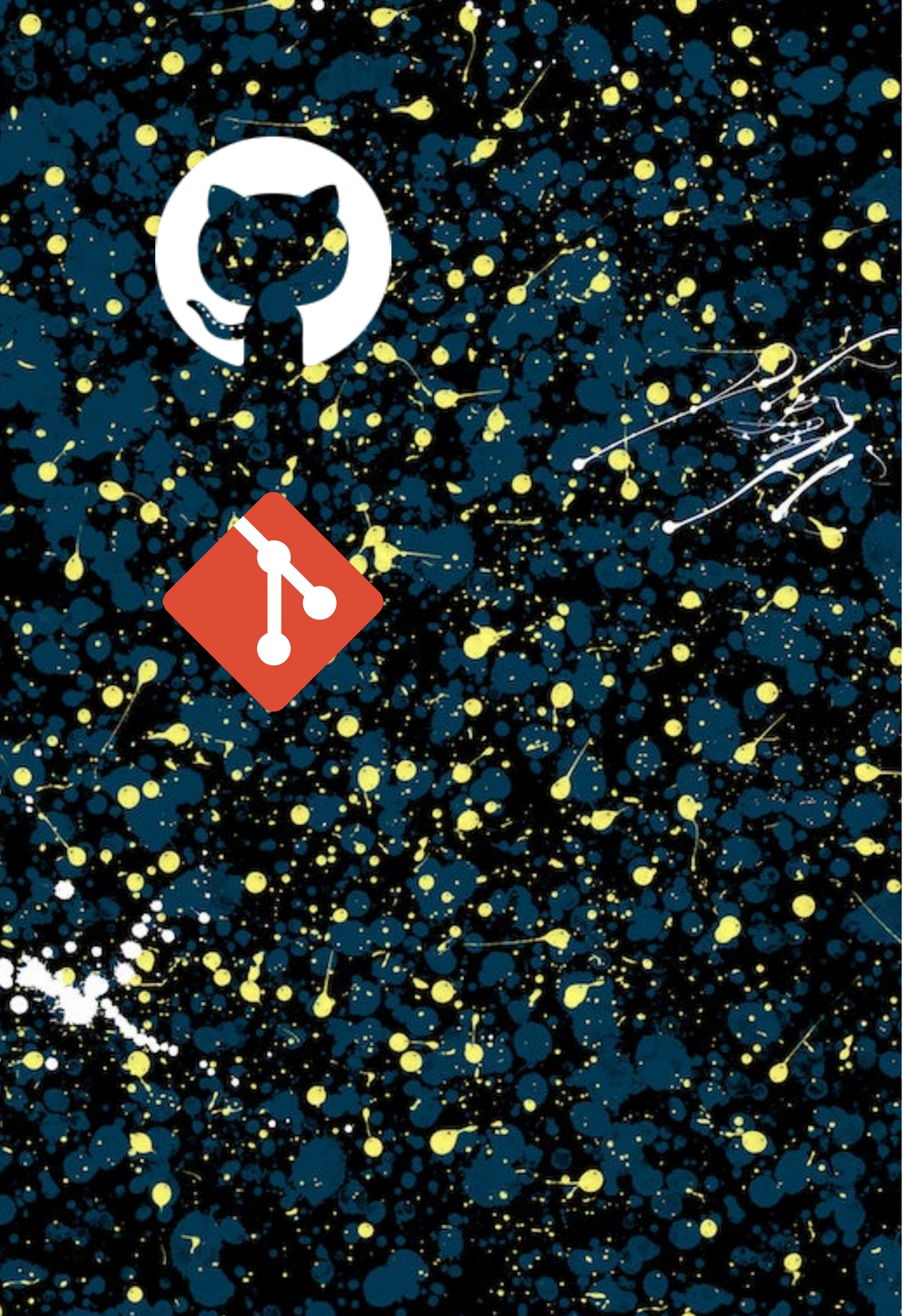
直接pushの禁止



設定内容 (抜粋)

- 保護対象のブランチ名
- マージ前のプルリクを必須
  - 最低1人の承認が必須

- 管理者も対象



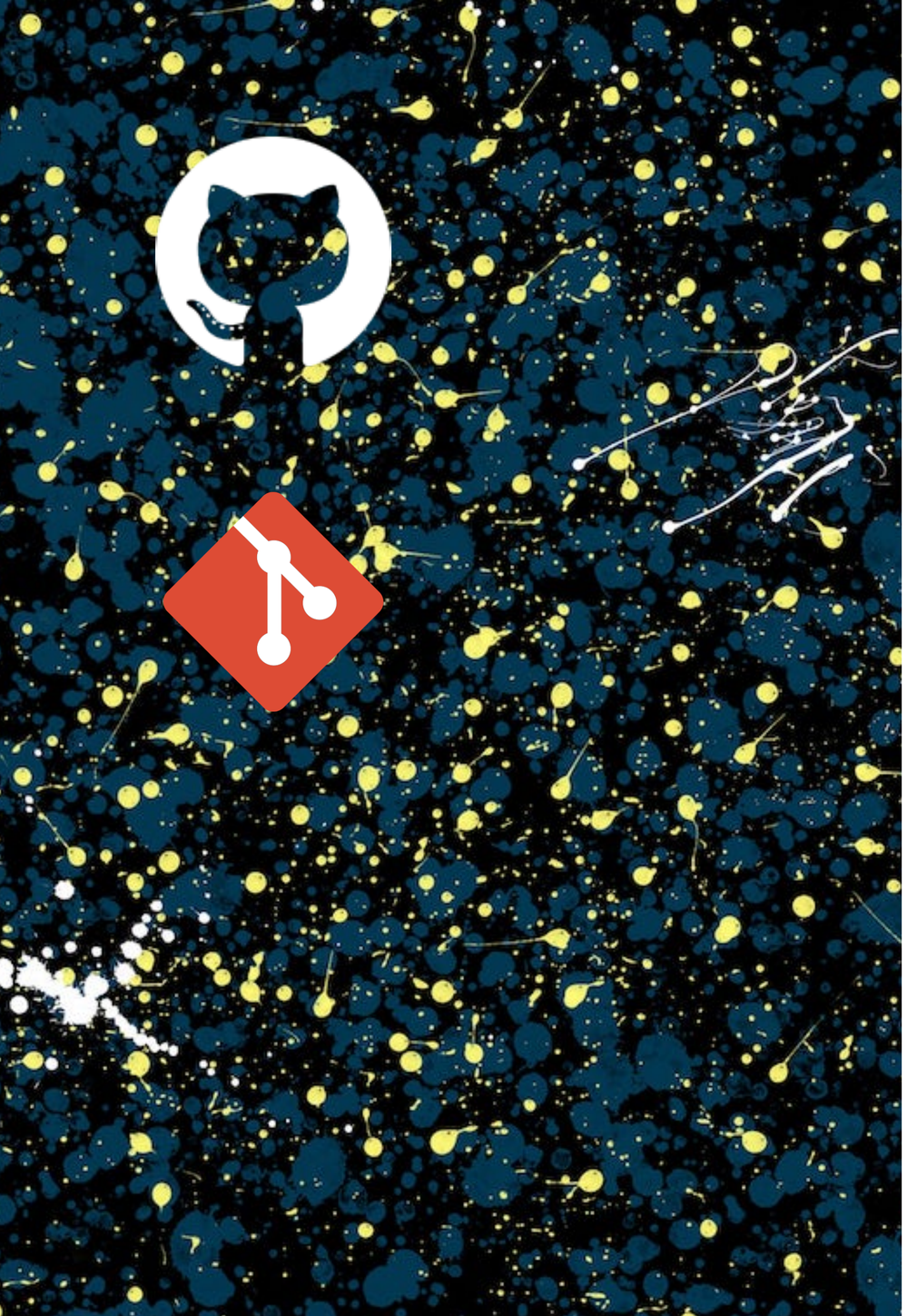
Freeプランと Proプランの 比較表

FreeとProの比較表

2022年5月 時点

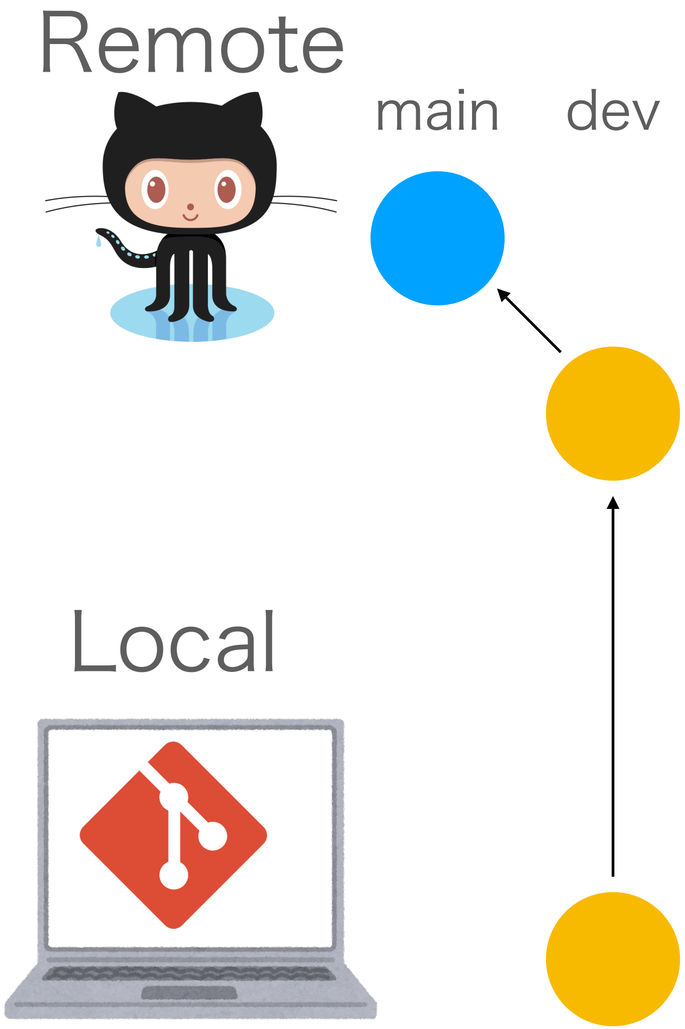
	Free	Pro
ブランチの保護	publicのみ	無制限
Pages and Wikis	publicのみ	無制限
コードオーナー (PR時自動レビュワー指定)	publicのみ	無制限
複数アサイン	publicのみ	無制限
複数レビュワー	publicのみ	無制限
Actions	public: 無制限 private: 2000分 / 月	public: 無制限 private 3000分 / 月
Packages	public: 無制限 private: 公開可能容量 500MB データ転送量 1GB / 月	public: 無制限 private: 公開可能容量 2GB データ転送量 10GB / 月

https://github.com/settings/billing/plans



プルリクの
テンプレート

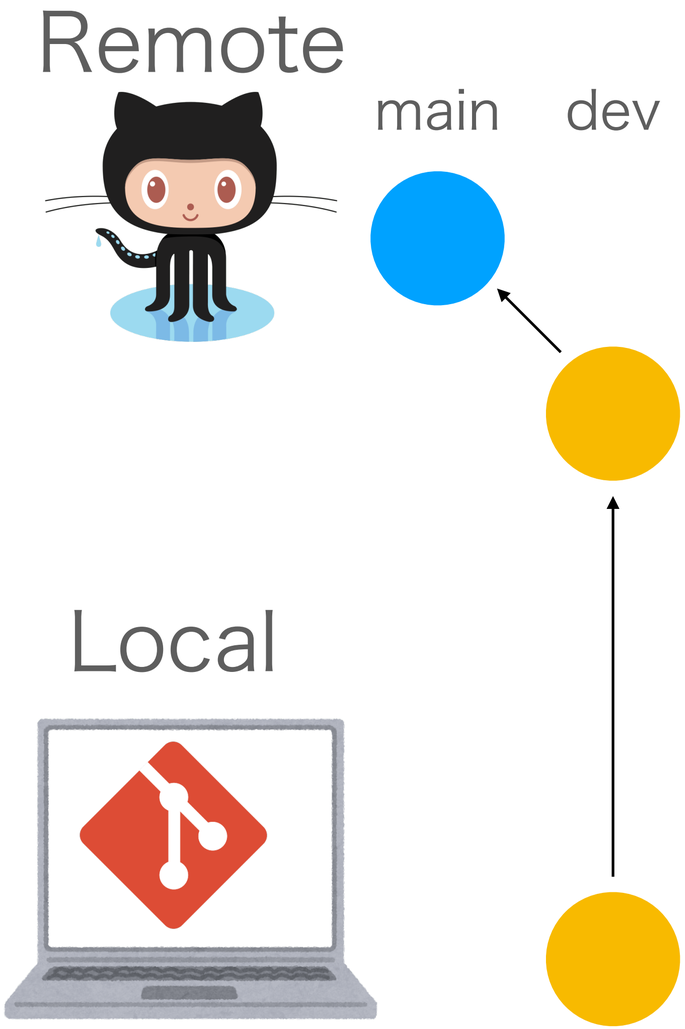
プルリクのテンプレート



第３者が見ると想定して
見やすい、レビューしやすい
内容にしておきたい

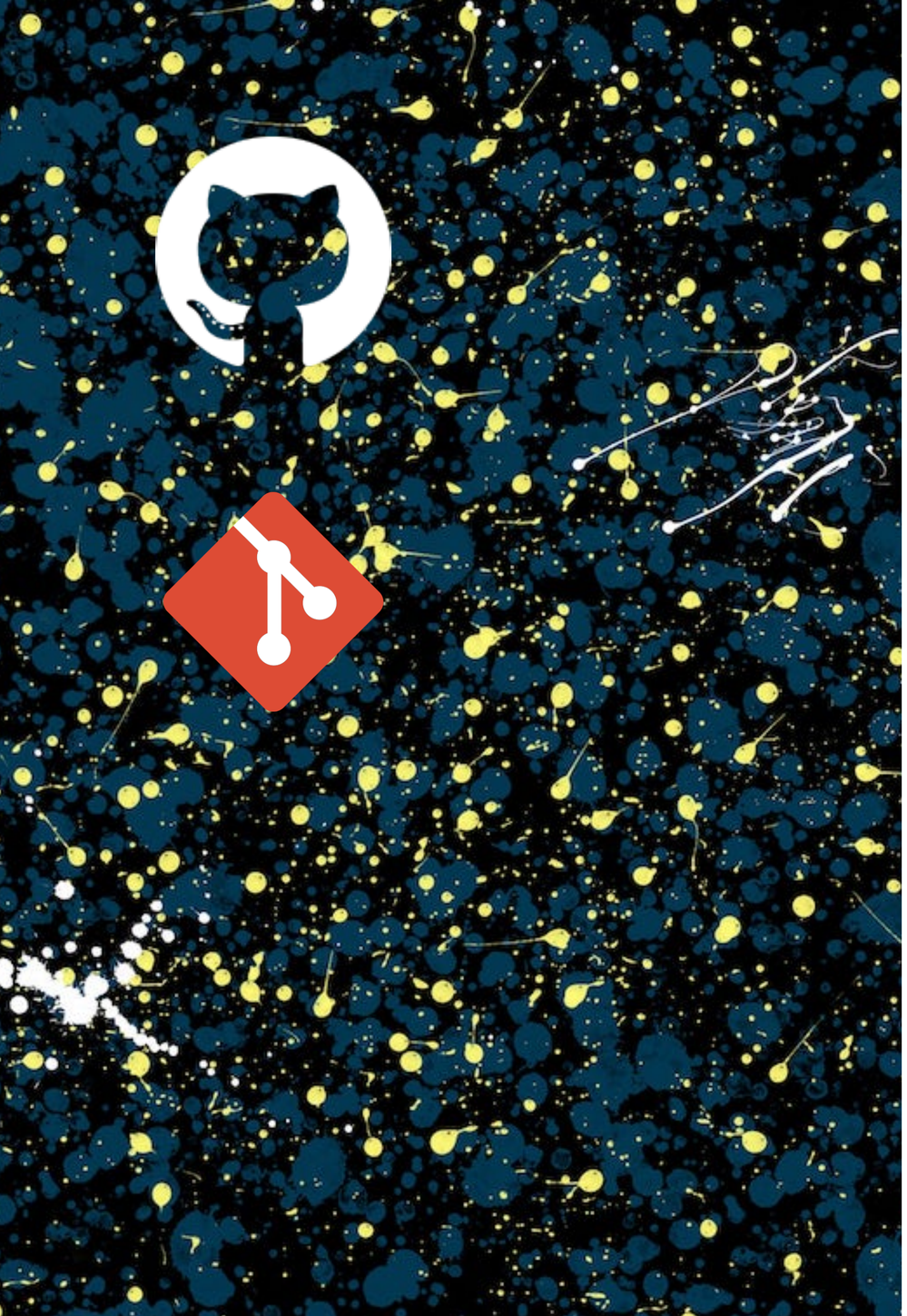
手順
https://docs.github.com/ja/communities/using-templates-to-encourage-useful-issues-and-pull-requests/creating-a-pull-request-template-for-your-repository

プルリクのテンプレート



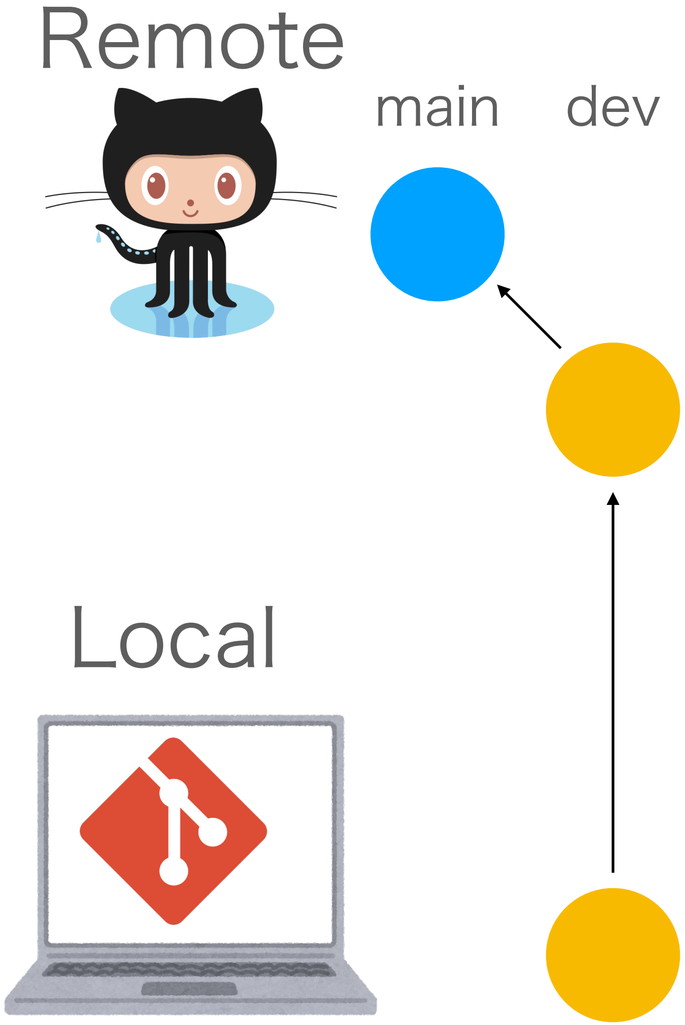
含めたい内容 (マークダウン) (参考までに)

```
### Why
- 実装背景

### What
- 実装内容
    - 実装内容詳細(その1)
    - 実装内容詳細(その1)

## Ref
- 関連PRへの参照リンク

## Check
-   [ ] レビュー依頼前に必ず実施する事(その1)
-   [ ] レビュー依頼前に必ず実施する事(その2)
```

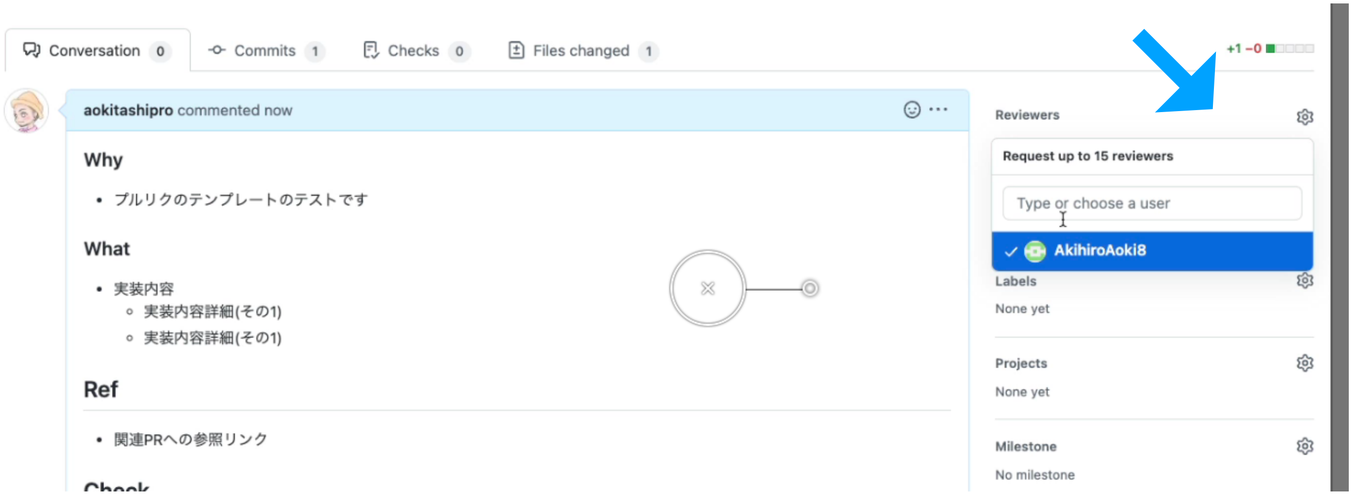


レビュー

プルリクのテンプレート



画面右のReviewers から
レビュー依頼する人を選択

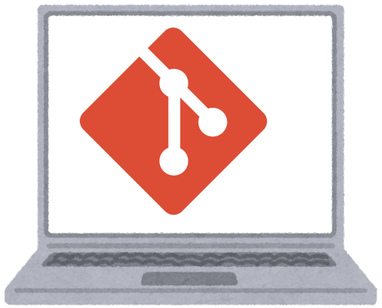


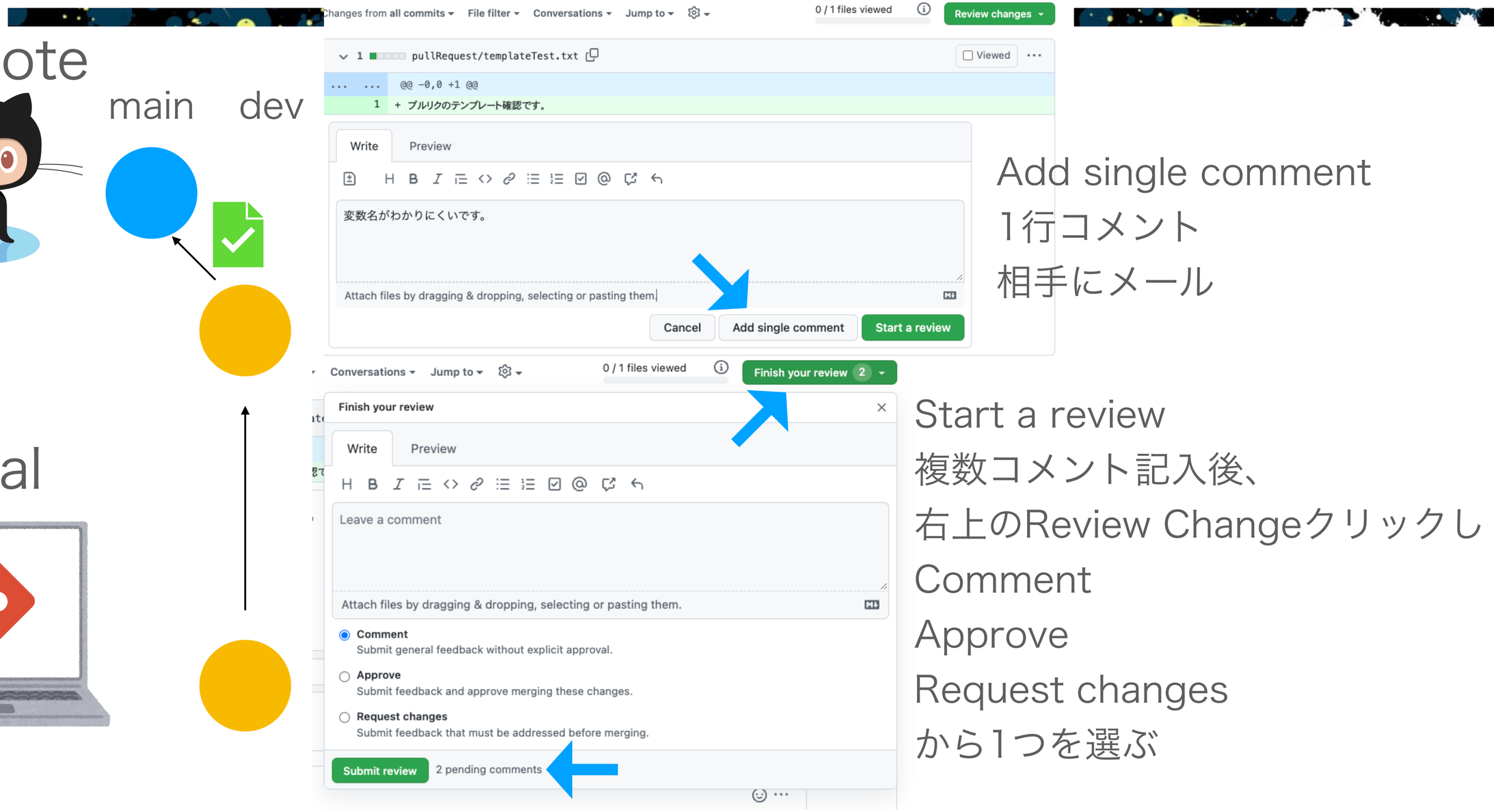
レビュー

Remote

main　dev

Local



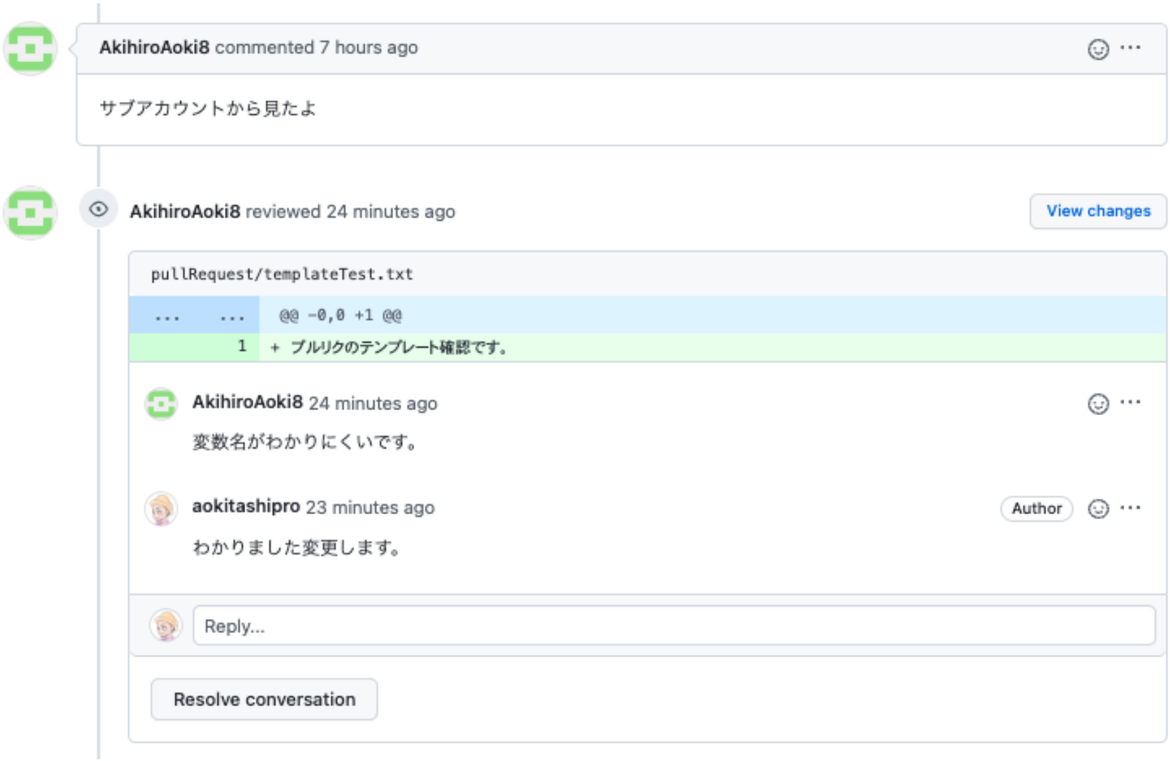


Add single comment
1行コメント
相手にメール

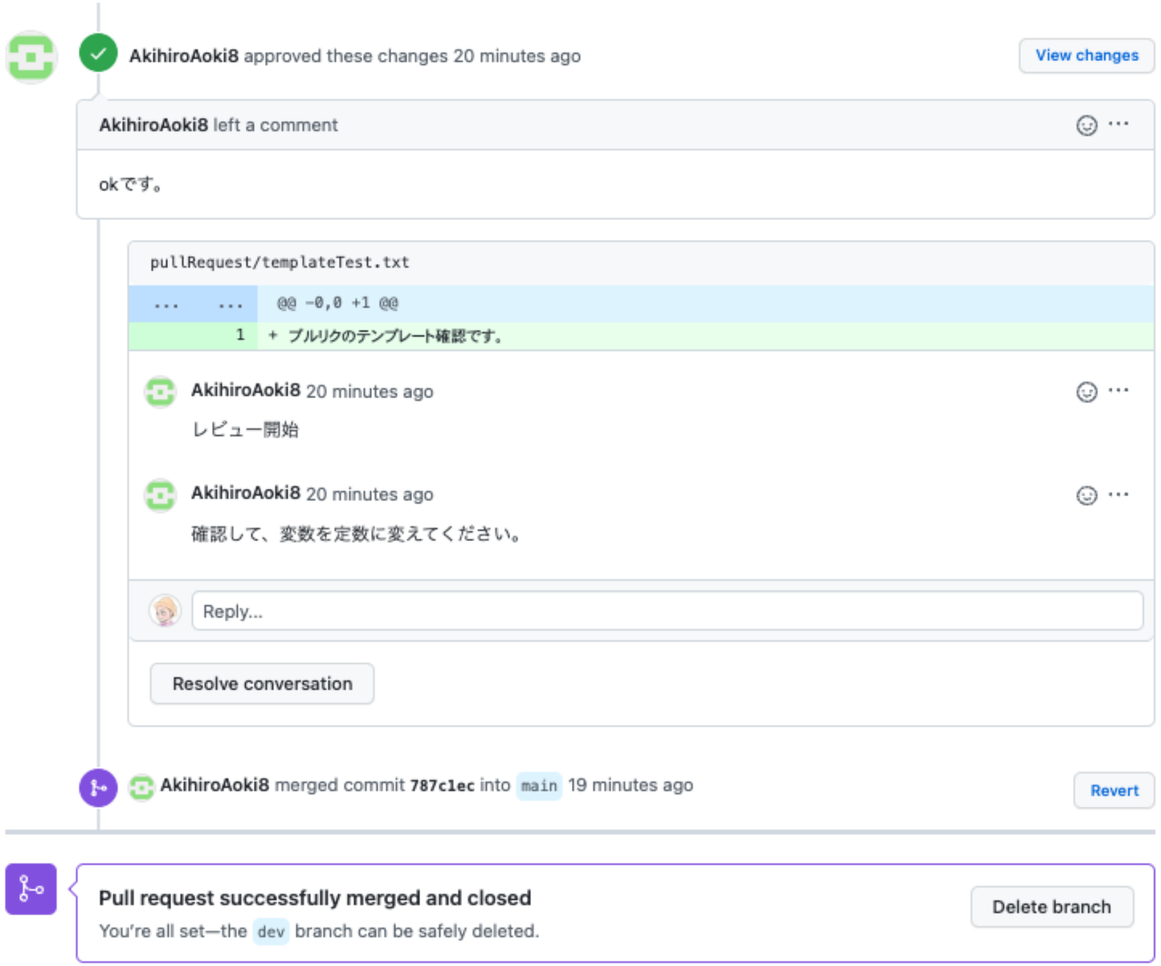
Start a review
複数コメント記入後、
右上のReview Changeクリックし
Comment
Approve
Request changes
から1つを選ぶ

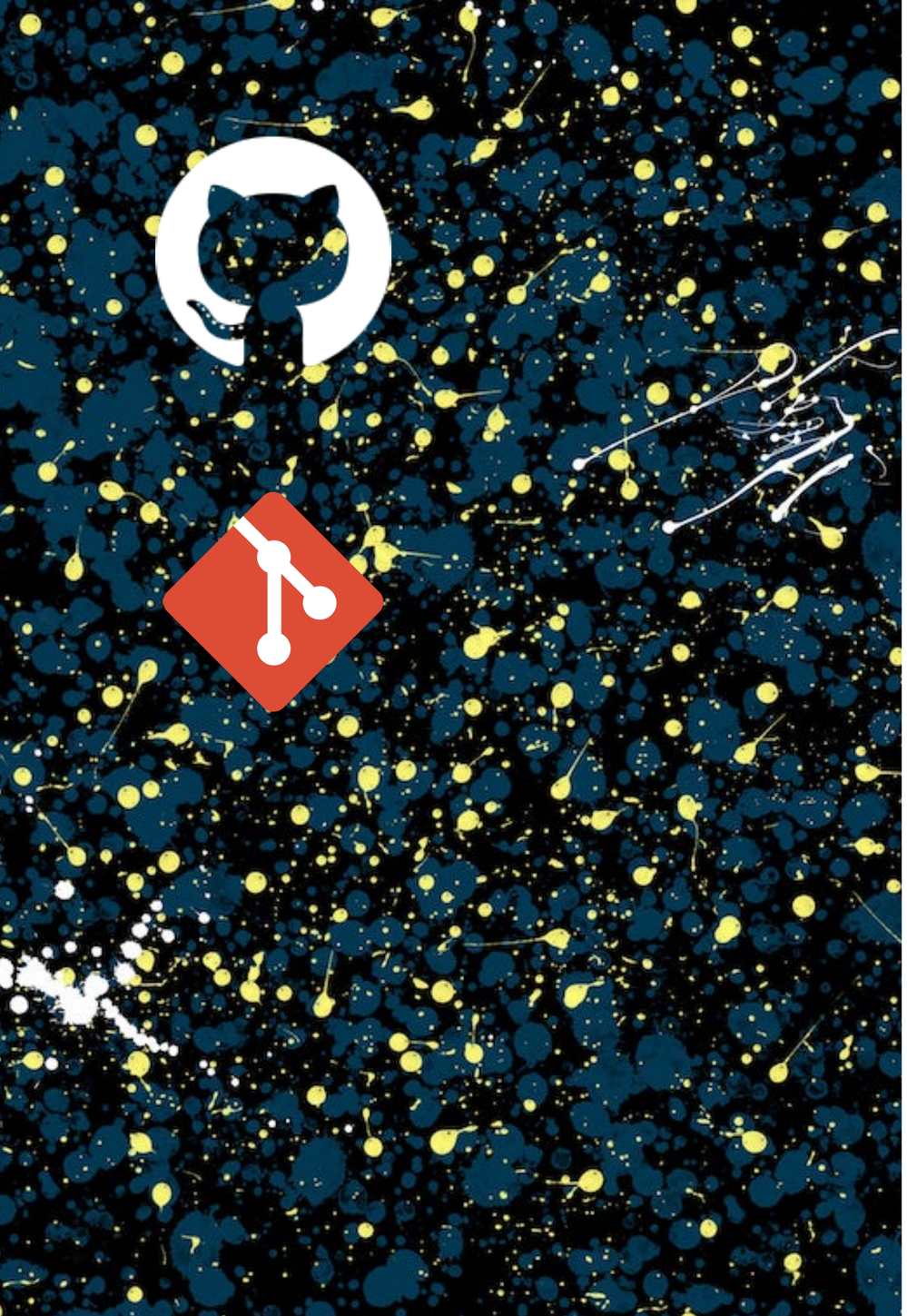
レビュー後のプルリク

1行コメント



複数コメント +Approve

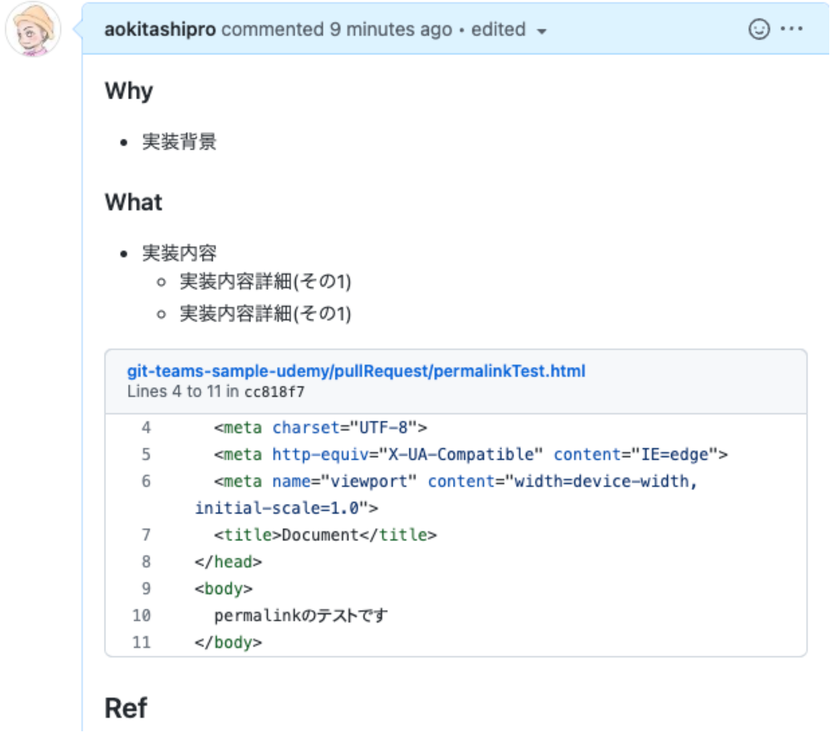


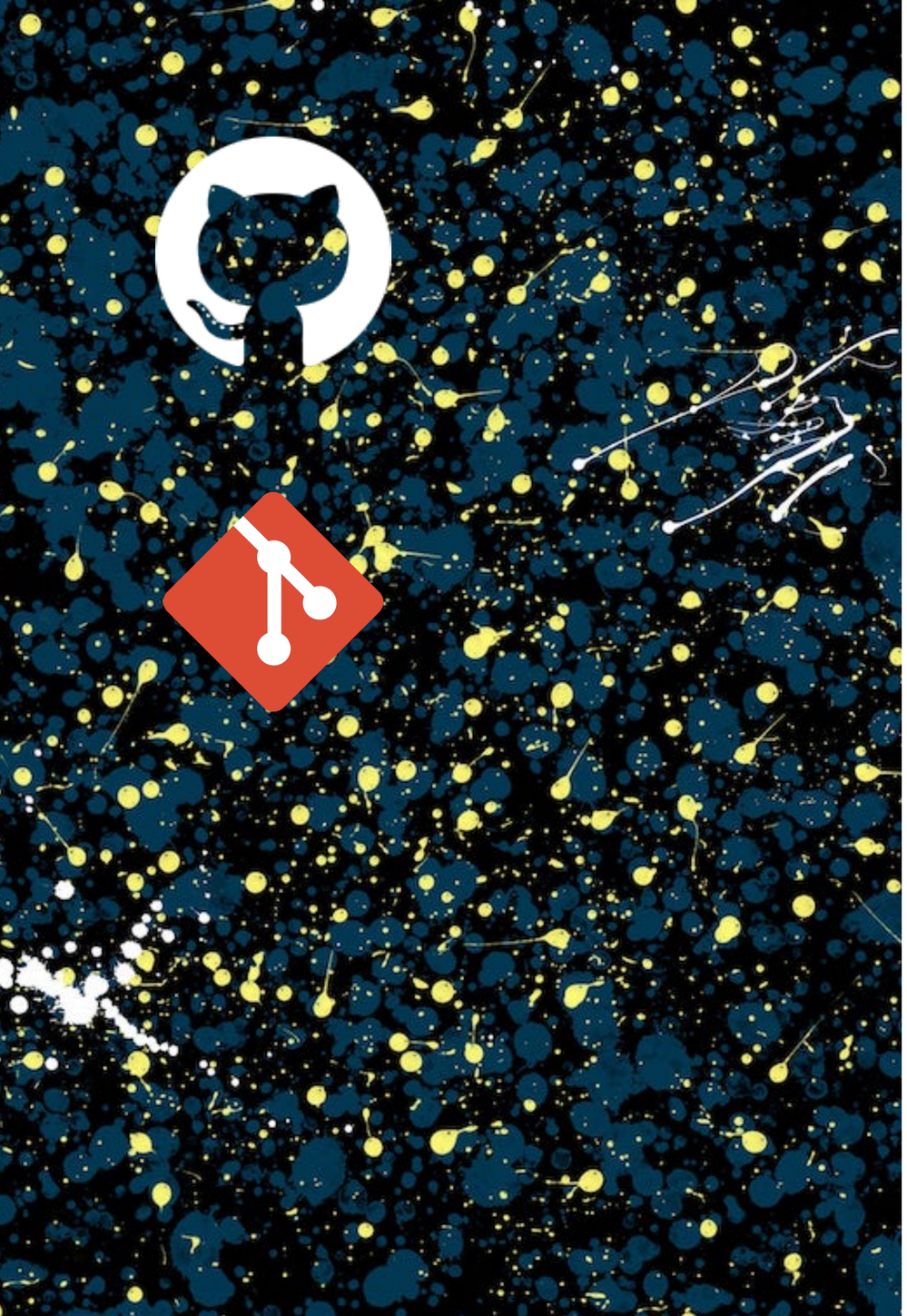


permalink機能

permalink

permanent(永続)
ブランチの変更に関わらずリンクが残る機能
コード選択し・・・マークをクリックで選択



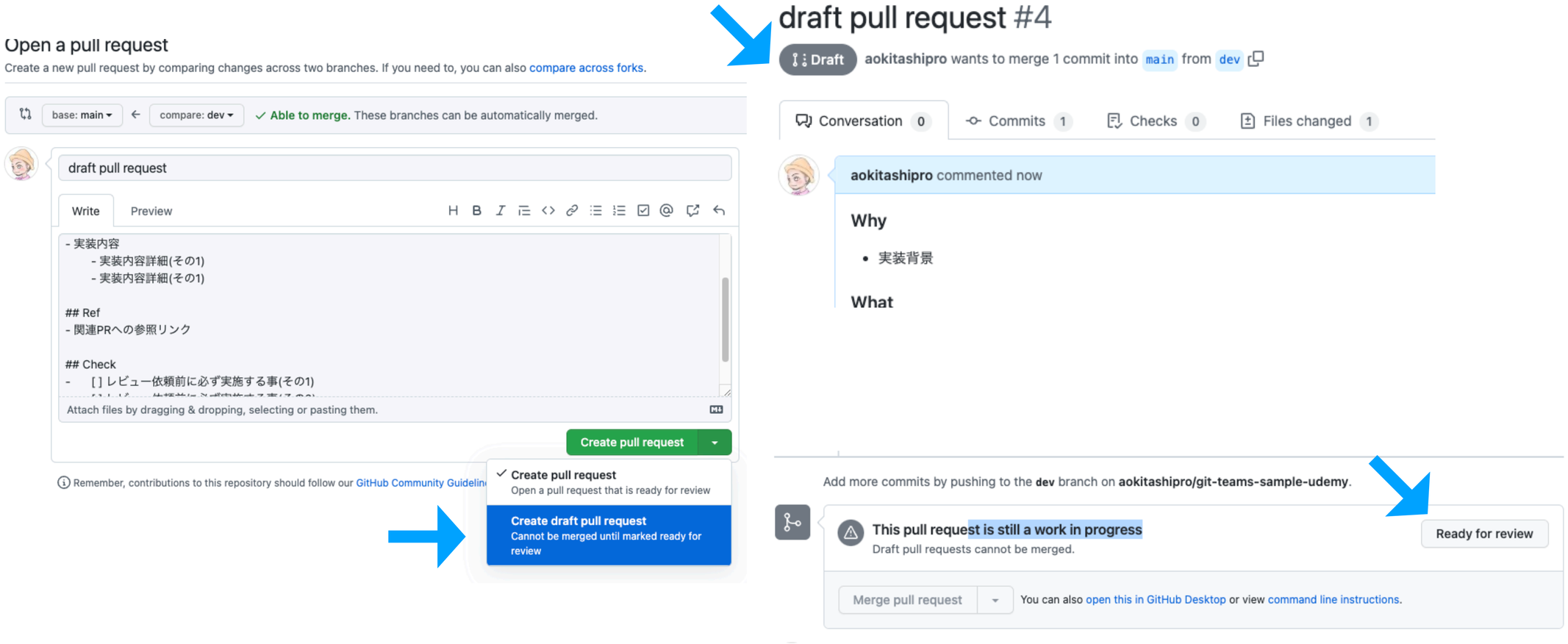


Draft
Pull Request

Draft Pull Request

PRをDraft(下書き)にすると、
作業中というのを明確にできる
(Draftでもレビュアー設定をすると
通知が飛ぶので注意)

パブリックで対応・プライベートで非対応
GitHub Free / GitHub Pro
パブリック・プライベート対応
GitHub Team / Enterprise Cloud / Enterprise Server

Draft Pull Request

PR作成時に draftを選べるようになる
準備できたら Ready for reviewを
クリックする事でマージ可能になる